

Dynamic Programming (D.P.)

It is a problem-solving technique that is used to perform **Recursion** **without Repetition** by storing the results of **unique sub-problems** in a table and referring to it as and when required.

Fibonacci Numbers

Given a non-negative number n , design a recursive algorithm to compute the n^{th} Fibonacci number.

n	0	1	2	3	4	5	6	7	8	9	...
$f(n)$	0	1	1	2	3	5	8	13	21	34	...

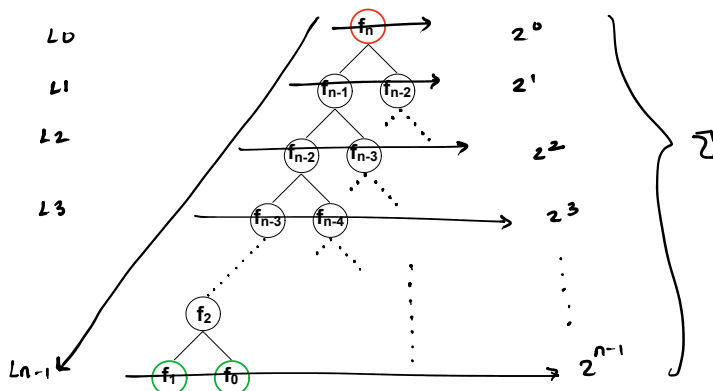
Recursive Case $f_1 = 1$

Let $f(n)$ be a function which computes the n^{th} Fibonacci number then

$$f(n) = f(n-1) + f(n-2)$$

Base Case

$$\begin{aligned} f(0) &= 0 \\ f(1) &= 1 \end{aligned}$$



Assuming all the levels are filled, total no. of sub-problems is equal to $2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^{n-1}$

This is a **finite GP** where,

$$\begin{aligned} a &= 1 \\ r &= 2 \\ \# &= n \end{aligned}$$

The sum of finite GP given as follows

$$= a * [(r^{\#}-1) / (r-1)] ,)$$

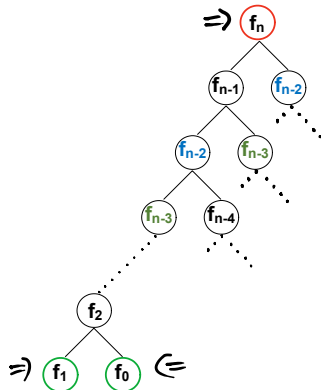
$\# = n$
 The sum of finite GP given as follows
 $= a * [(r^n - 1) / (r - 1)]$
 Substituting the values of a, r, and #, we get
 $= 1 * [(2^n - 1) / (2 - 1)]$
 $= 2^n - 1$
 $\sim 2^n$

How much time do we spend on each sub-problem ?

Therefore, the time complexity is equal to $O(2^n)$

What is the reason behind **exponential** time complexity ?

const Space: $O(n)$ due
 to fn
 call stack



1. find out no. of
unq. subproblems

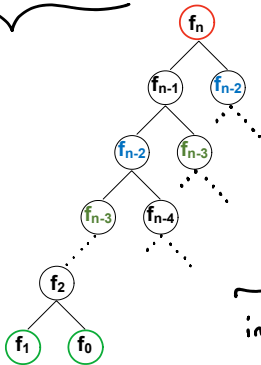
f_0 to f_n
 $n+1$ unq. subproblem

2. now we can create an
array (as array) of size
 $n+1$ to store results of
all the unique subproblems

lookup
 if (dp[n] != -1) {
 return dp[n];
 }

Top-Down Dynamic Programming

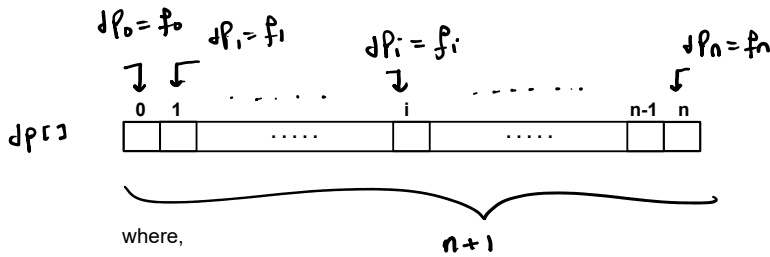
Memorization



useful during
 lookups
 init: -1

base case
 if (n == 0 || n == 1) {
 return dp[n] = n;
 }

recursive case
 return dp[n] = f(n-1) + f(n-2);



$f(i)$ is a function which computes the i^{th} Fibonacci number

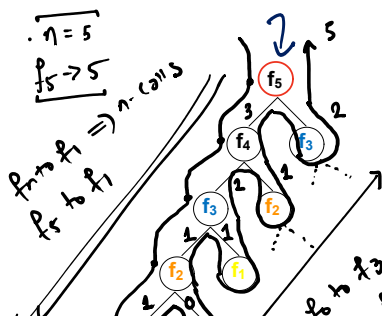
time: $(n+1) \cdot \text{const} \sim O(n)$

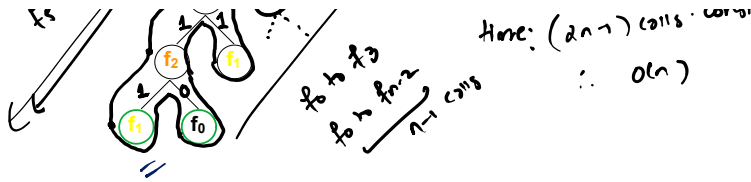
dp	0	1	2	3	4	5
	f_0	f_1	f_2	f_3	f_4	f_5
	0	1	1	2	3	5

=
 time: $(2n+1) \text{ calls} \cdot \text{const}$
 $\therefore O(n)$

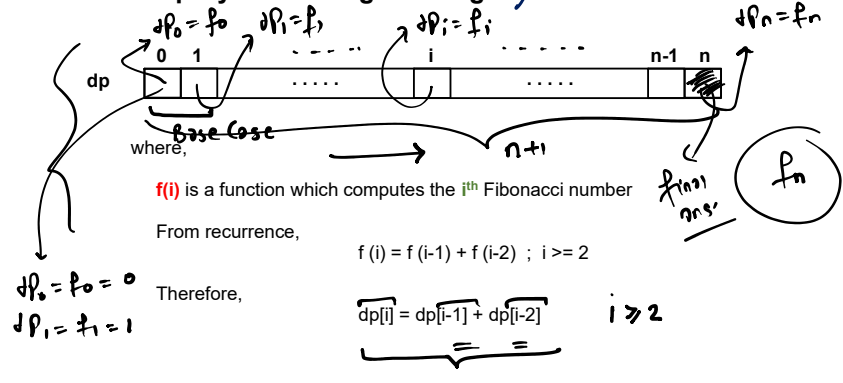
n due to
 fn call stack
 + n due
 to dp array

space: $O(n)$





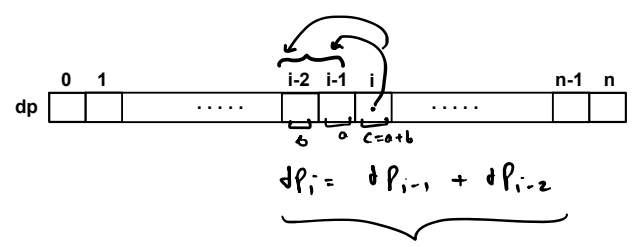
Bottom-Up Dynamic Programming / Tabulization



$$\begin{aligned} f_i &= f_{i-1} + f_{i-2} \\ \uparrow &\quad \downarrow \quad \downarrow \\ dp_i &= dp_{i-1} + dp_{i-2} \end{aligned}$$

$$f_n = f_{n-1} + f_{n-2}$$

Space Optimized Bottom-Up Dynamic Programming



Minimum Steps to One

Given a positive number n , design a recursive algorithm to compute the minimum no. of steps required to reduce the given no. to one such that at each step you can

- reduce n to $n-1$
- reduce n to $n/2$ if n is divisible by 2 or
- reduce n to $n/3$ if n is divisible by 3 or

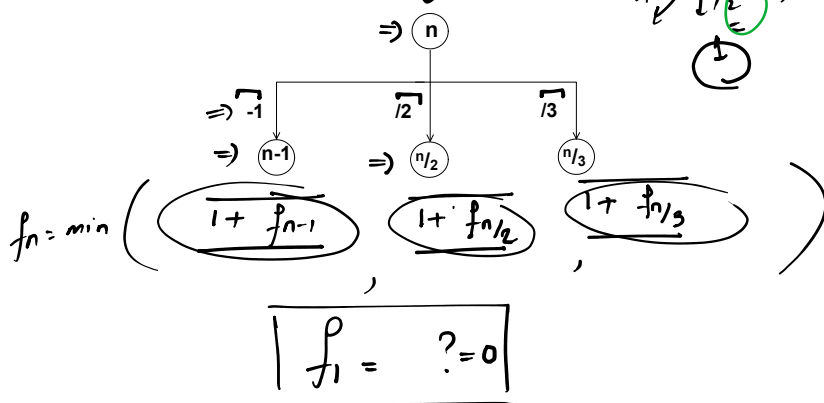
Example

Input : $n = 10$

Output : 3

Explanation : $10 \rightarrow 9 \xrightarrow{/3} 3 \xrightarrow{/3} 1$

$f(n)$ = find min steps needed to reduce n to 1

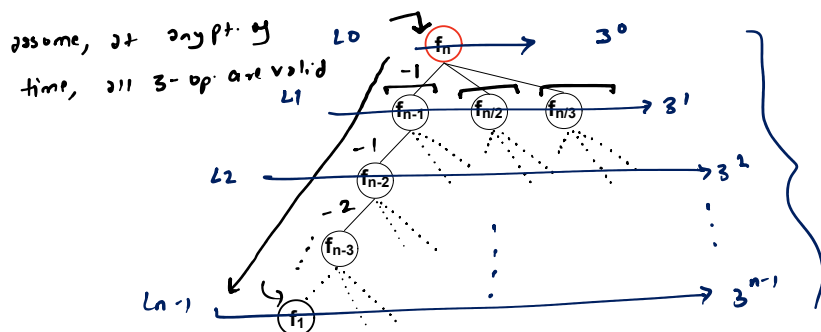


Recursive Case

Let $f(n)$ be a function which computes the minimum number of steps required to reduce n to one then

$$f(n) = 1 + \min \left(\begin{array}{l} f(n-1), \\ f(n/2), \\ f(n/3) \end{array} \right)$$

Base Case



Assuming all the levels are filled, total no. of sub-problems is equal to

$$3^0 + 3^1 + 3^2 + 3^3 + \dots + 3^{n-1}$$

This is a **finite GP** where,

$$\begin{aligned} a &= 1 \\ r &= 3 \\ \# &= n \end{aligned}$$

The sum of finite GP given as follows

$$= a * [(r^{\#}-1) / (r-1)]$$

Substituting the values of a, r, and #, we get

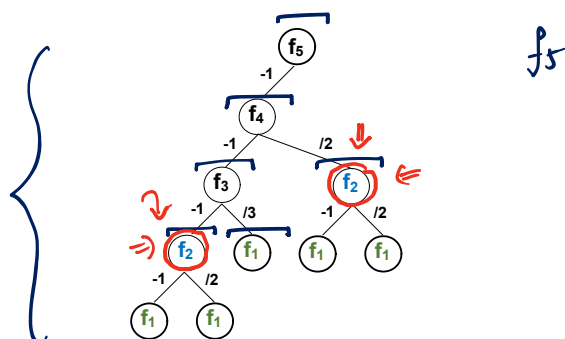
$$\begin{aligned} &= 1 * [(3^n-1) / (3-1)] \\ &= (3^n-1) / 2 \\ &\sim 3^n \end{aligned}$$

How much time do we spend on each sub-problem ?

const

Therefore, the time complexity is equal to **$O(3^n)$**

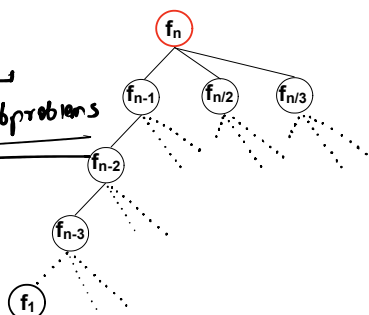
What is the reason behind **exponential** time complexity ?



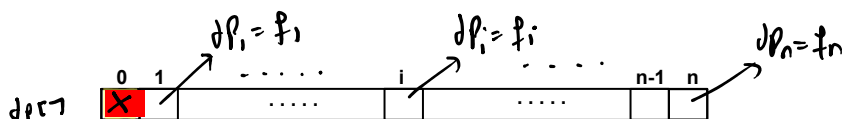
Top-Down Dynamic Programming

f_1 to f_n

n unique subproblems



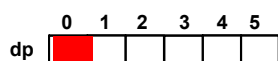
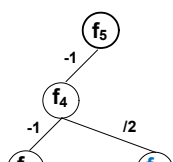
init: -1

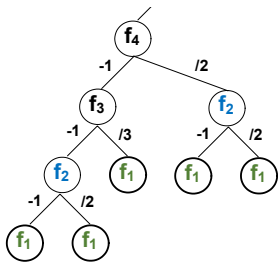


where,

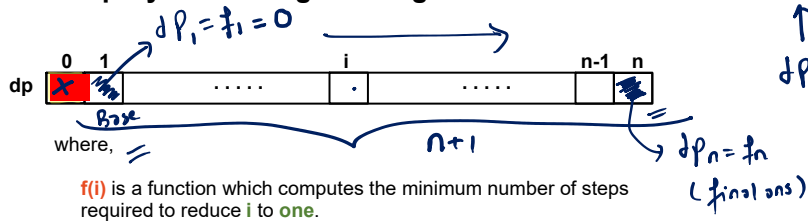
$n+1$

$f(i)$ is a function which computes the minimum number of steps required to reduce the i to **one**.





Bottom-Up Dynamic Programming



From recurrence,

$$f(i) = 1 + \min(f(i-1), f(i/2), f(i/3))$$

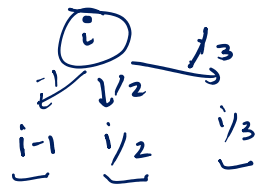
Therefore,

$$dp[i] = 1 + \min(dp[i-1], dp[i/2], dp[i/3])$$

$$f_i$$

$$\uparrow$$

$$dp_i =$$



Coin Change Problem

You are given an integer array **coins** representing coins of **K** different denominations and an integer **N** representing total amount of money.

Design a recursive algorithm to compute the fewest number of coins required to make up that amount i.e. reduce the given amount **N** to **0**.

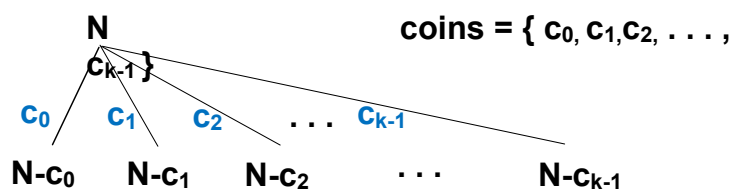
You may assume that you have an **infinite** supply of each kind of coin.

Example

Input : coins = [1, 2, 5], n = 11

Output : 3

Explanation : 11 = 5 + 5 + 1



Recursive Case

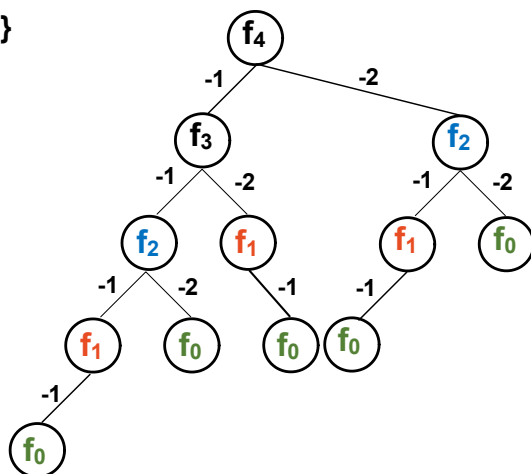
Let **f (n)** be a function which computes the minimum no. of coins required to reduce the given amount **N** to **0** then

$$f(n) = 1 + \min_{j=0}^{k-1} (f(n - c_j)) \quad ; \quad n - c_j \geq 0$$

Base Case

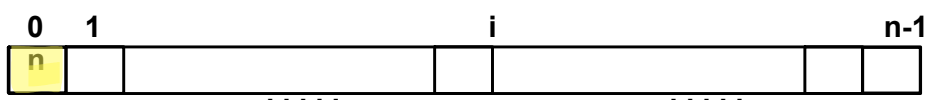
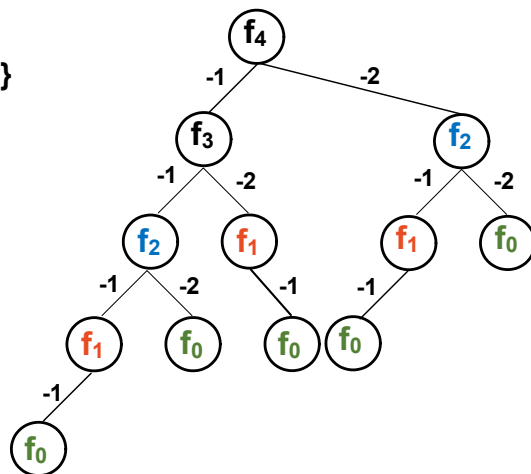
What is the reason behind **exponential** time complexity ?

N = 4
coins = { 1, 2 }



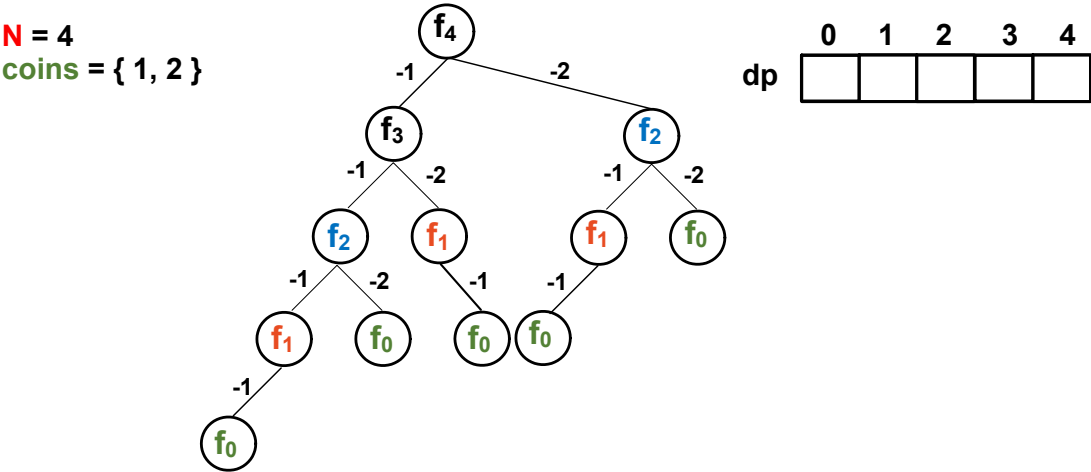
Top-Down Dynamic Programming

N = 4
coins = { 1, 2 }

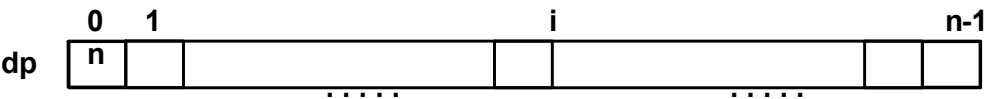


where,

f (i) is a function which computes the minimum number of coins required to reduce the amount **i** to **0**.



Bottom-Up Dynamic Programming



where,

f(i) is a function which computes the minimum number of coins required to reduce amount **i** to **0**.

From recurrence,

$$f(i) = 1 + \min_{j=0}^{k-1} (f(n-c_j)) \quad ; \quad i-c_j \geq 0$$

Therefore,

$$dp[i] = 1 + \min_{j=0}^{k-1} (dp[i-c_j])$$

Ladder's Problem

Consider a ladder with n steps, a person is standing at the bottom of the ladder and wants to reach to its top. Design a recursive algorithm to count the number of ways the person can reach the n^{th} step i.e. ladder's top from its bottom such that at each step the person can take a jump of size ranging b/w one to k steps.

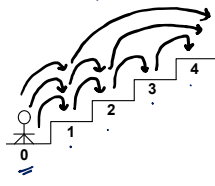
Example $\text{cond} = [1, 2, 3]$

Input: $n=4, k=3$

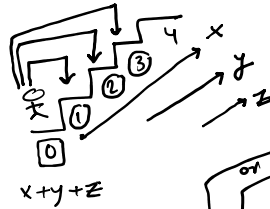
Output: 7

Explanation: $[1, 1, 1, 1]$

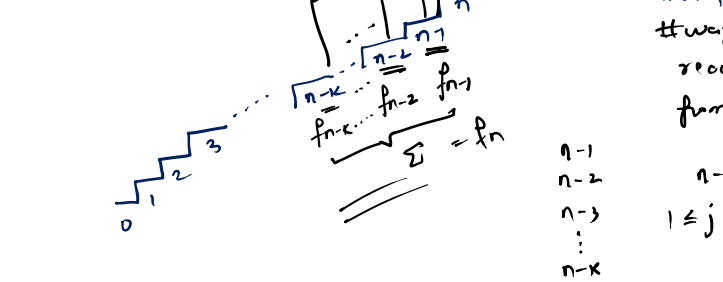
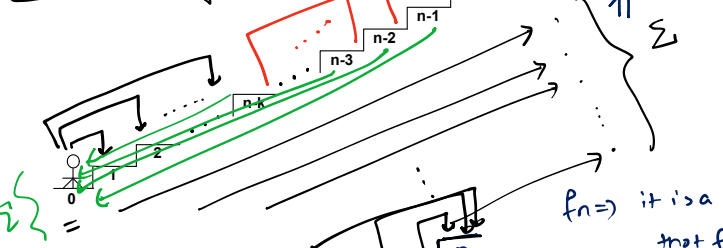
$[1, 1, 2]$
 $[1, 2, 1]$
 $[2, 1, 1]$
 $[2, 2]$
 $[1, 3]$
 $[3, 1]$



$k=3$ $\textcircled{1} 2, 3$



find #ways to go from 0 to n $\Rightarrow$ find #ways to reach n from bottom



$f_n \Rightarrow$ it is a fn that finds #ways to reach n from bottom

$n-1$
 $n-2$
 $n-3$
 $\vdots$
 $n-k$

$k=3$
 $1, 2, 3$

#ways to go from 0 to n = #ways to reach n from 0

$$f_n = \sum_{j=1}^k f_{n-j}$$

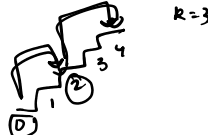
Recursive Case

Let $f(n)$ be a function which computes the no. of ways a person can reach the n^{th} step i.e. the top of the ladder from its bottom then,

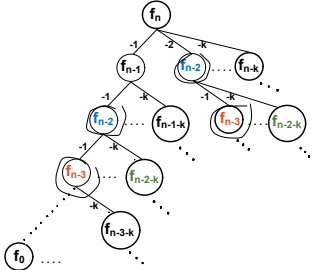
$$f(n) = \sum_{j=1}^k f(n-j) ; n-j \geq 0$$

Base Case

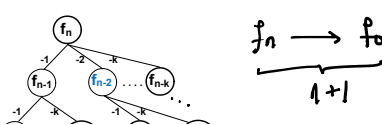
$$f(0) = ? = 1$$



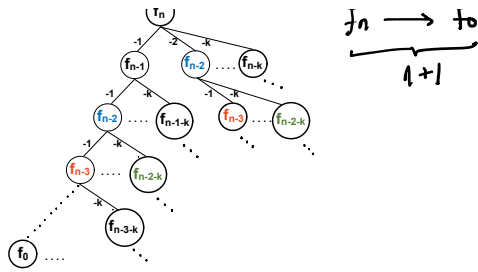
What is the reason behind exponential time complexity?



Top-Down Dynamic Programming

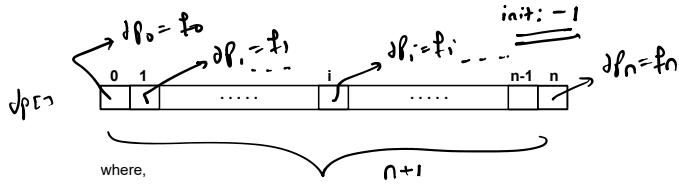


$$f_n \rightarrow f_0$$



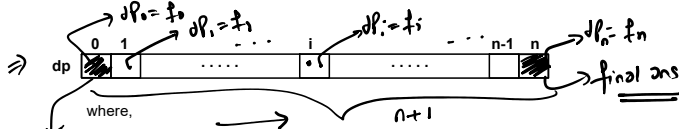
$$f_n \rightarrow f_0$$

$$1+1$$



where, $f(i)$ is a function which computes the total number of ways a person can reach the i^{th} step of the ladder from its bottom.

Bottom-Up Dynamic Programming



Base case
From recurrence,

$$dp_0 = f_0 = 1$$

$$f(i) = \sum_{j=1}^k f(i-j) \quad ; i-j \geq 0, i > 0$$

Therefore,

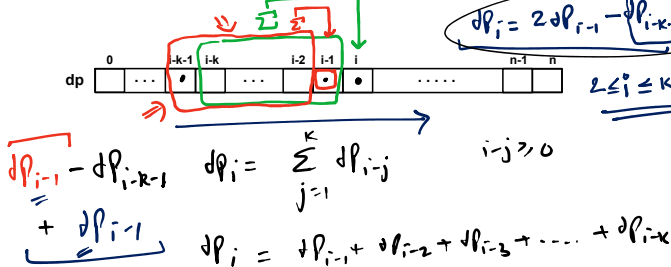
$$dp[i] = \sum_{j=1}^k dp[i-j]$$

$$f_n = \sum_{j=1}^k f_{n-j} \quad n-j \geq 0$$

$$f_i = \sum_{j=1}^k f_{i-j} \quad i-j \geq 0$$

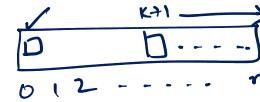
$$dp_i = f_i$$

Time Optimized Bottom-Up Dynamic Programming



$$dp_i = \sum_{j=1}^k dp_{i-j}$$

$$dp_i = dp_{i-1} + dp_{i-2} + dp_{i-3} + \dots + dp_{i-k}$$



$$\Rightarrow dp_0 = 1$$

$$\Rightarrow dp_1 = dp_0 = 1$$

$$dp_2 = dp_1 + dp_0 = 2dp_1$$

$$dp_3 = dp_2 + dp_1 + dp_0 = 2dp_2$$

$$\vdots$$

$$dp_i = 2dp_{i-1}$$

state

transition

House Robber I

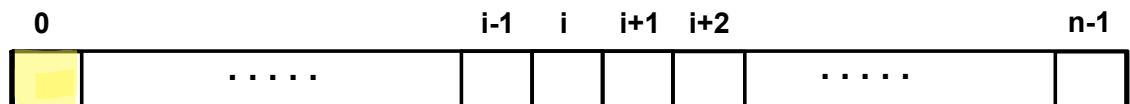
You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and **it will automatically contact the police if two adjacent houses were broken into on the same night.**

Given an integer array `nums` representing the amount of money of each house, return *the maximum amount of money you can rob tonight **without alerting the police.***

Example

Input : N = 5

0	1	2	3	4
100	50	400	200	100



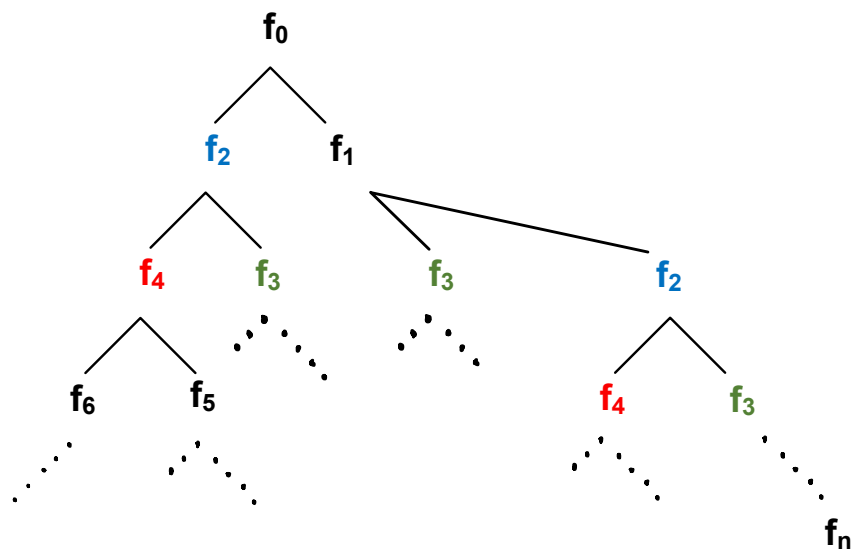
Recursive Case

Let $f(i)$ be a function which computes the **maximum** amount of money that can be robbed in a night from a given sequence of houses starting at the i^{th} **index** without alerting the police then,

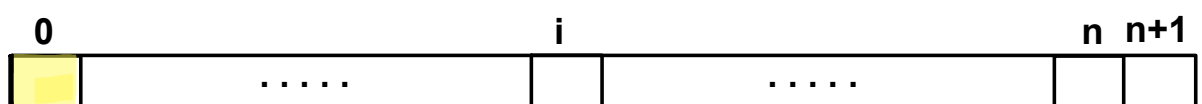
$$f(i) = \max \{ \text{nums}[i] + f(i+2), f(i+1) \}$$

Base Case

What is the reason behind **exponential** time complexity ?



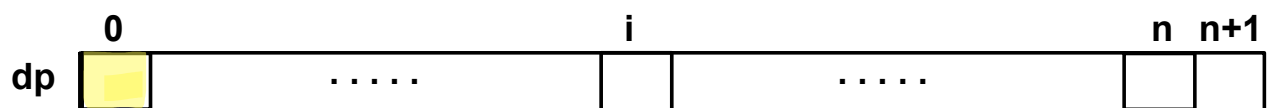
Top-Down Dynamic Programming



where,

f (i) is a function which computes the **maximum** amount of money which can be robbed in a night from a given sequence of houses starting at the **ith index** without alerting the police.

Bottom-Up Dynamic Programming



From recurrence,

$$f(i) = \max \{ \text{nums}[i] + f(i+2), f(i+1) \}$$

Therefore,

$$\text{dp}[i] = \max \{ \text{nums}[i] + \text{dp}[i+2], \text{dp}[i+1] \}$$

House Robber II

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are **arranged in a circle**. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have a security system connected, and **it will automatically contact the police if two adjacent houses were broken into on the same night**.

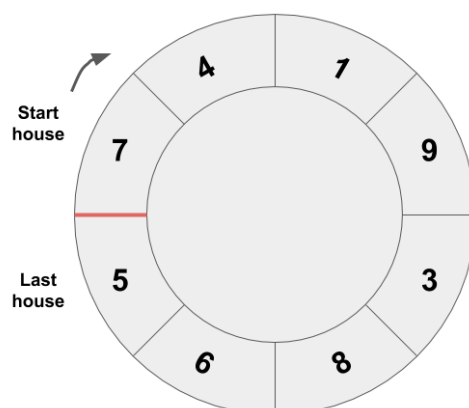
Given an integer array `nums` representing the amount of money of each house, return *the maximum amount of money you can rob tonight **without alerting the police***.

Example

Input : N = 8

0	1	2	3	4	5	6	7
7	4	1	9	3	8	6	5

0	1	2	3	4	5	6	7
7	4	1	9	3	8	6	5



Maximum value obtainable from

7	4	1	9	3	8	6
---	---	---	---	---	---	---

OR

4	1	9	3	8	6	5
---	---	---	---	---	---	---

House Robber III

Given the **root** of a binary tree, in which each **node** corresponds to a **house** and the **value of the node** represents the **amount of money** in each house. Design an algorithm, to find the **maximum** amount of money a thief can rob **without alerting** the police such that he is initially standing at the root node & police will be informed if two **directly-linked** houses are robbed together.

